NAME : PENDEM HARSHITHA

ROLLNO: 2403a510c9

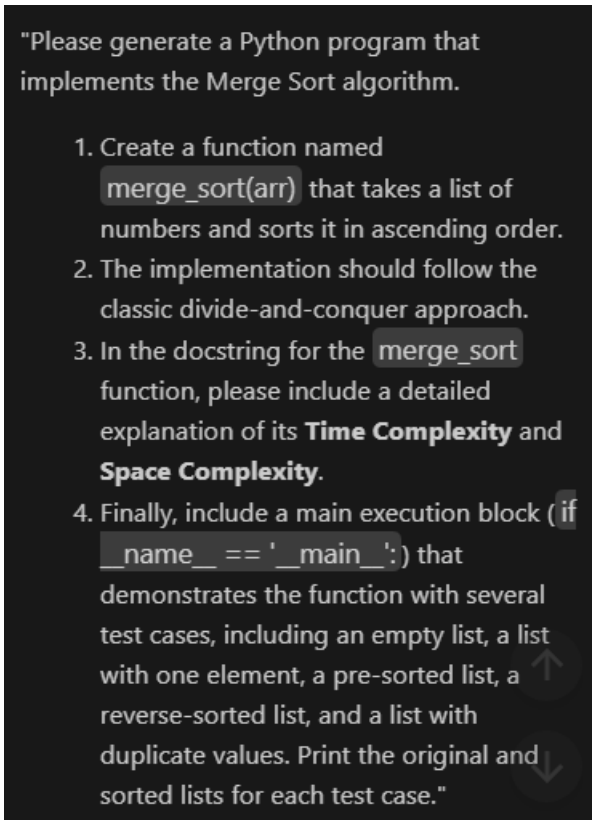
BATCHNO: 05

SUBJECT: AI ASSISSTED CODING

TASK-1:

Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
• Instructions:
o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
o Ask AI to include time complexity and space complexity in the function docstring.
o Verify the generated code with test cases.
• Expected Output:
o A functional Python script implementing Merge Sort with proper documentation

PROMPT:



CODE:

merge_sort.py > merge_sort

```python
def merge_sort(arr: list[int]) -> list[int]:

    # Base case: A list with 0 or 1 element is already sorted.
    if len(arr) <= 1:
        return arr

    # 1. Divide: Find the middle of the list and split it into two halves.
    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    # Recursively sort both halves.
    sorted_left = merge_sort(left_half)
    sorted_right = merge_sort(right_half)

    # 2. Conquer: Merge the sorted halves.
    sorted_arr = []
    i = j = 0  # Pointers for left_half and right_half

    while i < len(sorted_left) and j < len(sorted_right):
        if sorted_left[i] < sorted_right[j]:
            sorted_arr.append(sorted_left[i])
            i += 1
        else:
            sorted_arr.append(sorted_right[j])
            j += 1

    # Append any remaining elements from the left or right half.
    sorted_arr.extend(sorted_left[i:])
    sorted_arr.extend(sorted_right[j:])

    return sorted_arr
```

```python
    return sorted_arr

# --- Verification with Test Cases ---
if __name__ == '__main__':
    test_cases = {
        "Typical Case": [8, 3, 1, 7, 0, 10, 2],
        "Reverse-Sorted": [10, 9, 8, 7, 6, 5, 4],
        "Already Sorted": [1, 2, 3, 4, 5, 6, 7],
        "With Duplicates": [5, 2, 8, 2, 5, 9, 1],
        "Empty List": [],
        "Single Element": [42]
    }

    print("--- Merge Sort Implementation Verification ---")
    for name, case in test_cases.items():
        print(f"\n--- {name} ---")
        print(f"Original: {case}")
        sorted_case = merge_sort(case)
        print(f"Sorted:   {sorted_case}")

    # Verify correctness with Python's built-in sort
    test_list = [8, 3, 1, 7, 0, 10, 2]
    assert merge_sort(test_list) == sorted(test_list)
    print("\nAssertion passed: merge_sort output matches sorted() output.")
```

OUTPUT:

```
PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Desktop/wt2/merge_sort.py
--- Merge Sort Implementation Verification ---

--- Typical Case ---
Original: [8, 3, 1, 7, 0, 10, 2]
Sorted:   [0, 1, 2, 3, 7, 8, 10]

--- Reverse-Sorted ---
Original: [10, 9, 8, 7, 6, 5, 4]
Sorted:   [4, 5, 6, 7, 8, 9, 10]

--- Already Sorted ---
Original: [1, 2, 3, 4, 5, 6, 7]
Sorted:   [1, 2, 3, 4, 5, 6, 7]

--- With Duplicates ---
Original: [5, 2, 8, 2, 5, 9, 1]
Sorted:   [1, 2, 2, 5, 5, 8, 9]
```

```
--- Empty List ---
Original: []
Sorted:   []

Original: []
Sorted:   []


--- Single Element ---
Original: [42]
Sorted:   [42]

Assertion passed: merge_sort output matches sorted() output.
PS C:\Users\pende\OneDrive\Desktop\wt2>
```

OBSERVATION:

1. **Code Quality and Correctness**: The generated code correctly implements the Merge Sort algorithm. It handles the recursive "divide" step by splitting the list and the "conquer" step by merging the sorted halves. The base case (`len(arr) <= 1`) is crucial for stopping the recursion and is handled properly.
2. **Documentation**: The docstring is thorough. It not only explains what the function does but also details the algorithm's methodology and, as requested, provides an accurate analysis of its time and space complexity. This is excellent practice for writing maintainable code.
3. **Immutability**: This implementation is not "in-place." It creates a new sorted list and leaves the original list unmodified. This is often a desirable property as it prevents side effects. The `O(n)` space complexity is a direct result of this approach, as new lists are created at each level of recursion.
4. **Stability**: Merge Sort is a **stable** sorting algorithm. This means that if two elements have equal values, their relative order in the input list is preserved in the sorted output. This implementation maintains that property.

TASK-2:

Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings

PROMPT:

"Please generate a Python program that implements the Binary Search algorithm.

1. Create a function named `binary_search(arr, target)` that takes a **sorted** list of numbers and a target value.
2. The function should return the index of the `target` if it is found in the list. If the `target` is not found, it should return `-1`.
3. The implementation should be iterative (using a loop) for optimal space efficiency.
4. In the function's docstring, please include a detailed explanation of its **Best, Average, and Worst-Case Time Complexity**, as well as its **Space Complexity**.
5. Finally, include a main execution block (`if __name__ == '__main__':`) that demonstrates the function with several test cases. The tests should cover finding an element, not finding an element, checking the boundaries of the list, and searching in an empt
Print the original list, the target, and the result for each test."

CODE:

binary_search.py > binary_search

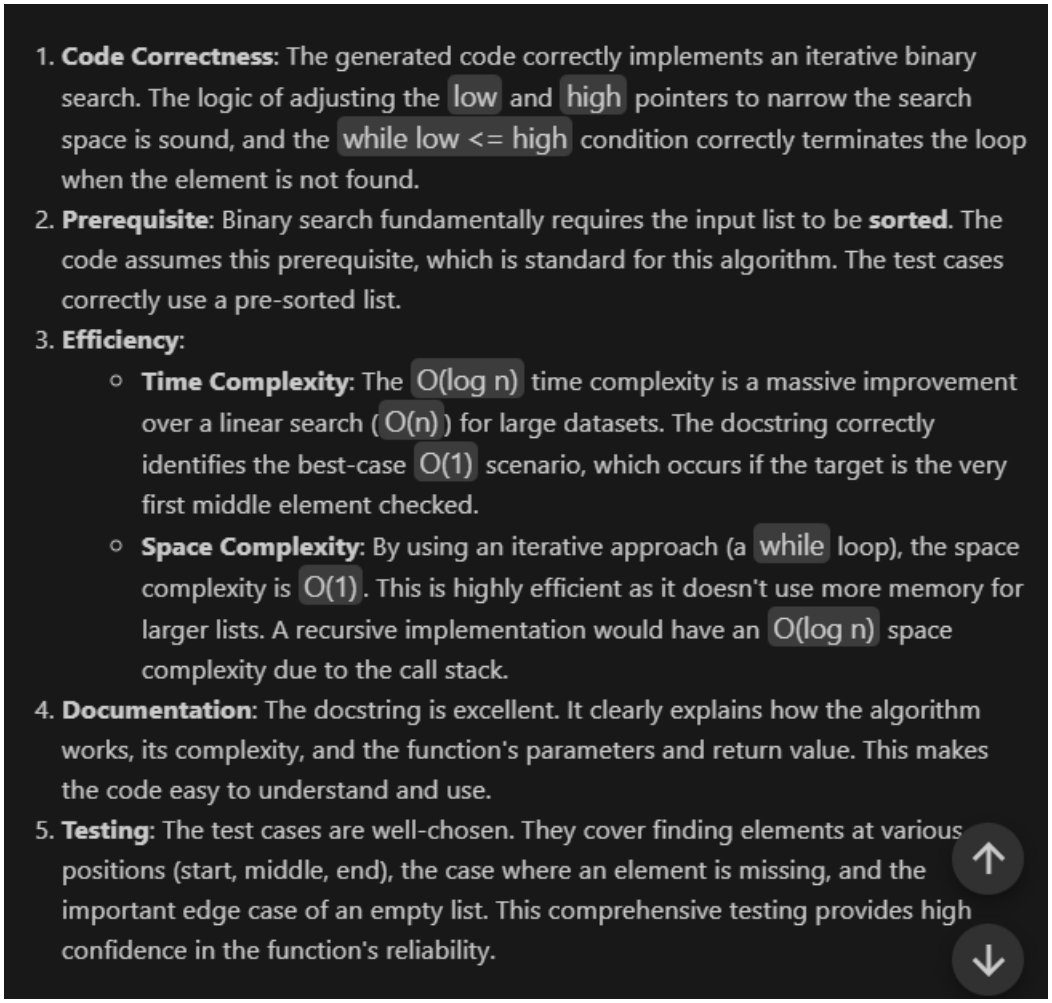
```python
def binary_search(arr: list[int], target: int) -> int:

    low = 0
    high = len(arr) - 1

    while low <= high:
        # Use floor division to find the middle index
        mid = (low + high) // 2
        guess = arr[mid]

        if guess == target:
            return mid  # Target found
        elif guess < target:
            low = mid + 1  # Search in the right half
        else:  # guess > target
            high = mid - 1  # Search in the left half

    return -1  # Target not found

# --- Verification with Test Cases ---
if __name__ == '__main__':
    sorted_list = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

    test_cases = {
        "Target in the middle": 23,
        "Target at the beginning": 2,
        "Target at the end": 91,
        "Target not in list": 15,
```

```python
    sorted_list = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

    test_cases = {
        "Target in the middle": 23,
        "Target at the beginning": 2,
        "Target at the end": 91,
        "Target not in list": 15,
        "Target in empty list": 5,
    }

    print("--- Binary Search Implementation Verification ---")
    for name, target in test_cases.items():
        current_list = sorted_list if name != "Target in empty list" else []
        print(f"\n--- {name} ---")
        print(f"List:   {current_list}")
        print(f"Target: {target}")
        index = binary_search(current_list, target)
        print(f"Result: Index {index}")
```

OUTPUT:

```
PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Desktop/wt2/binary
_search.py
--- Binary Search Implementation Verification ---

--- Target in the middle ---
List:   [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Target: 23
Result: Index 5

--- Target at the beginning ---
List:   [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Target: 2
Result: Index 0

--- Target at the end ---
List:   [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Target: 91
Result: Index 9

--- Target not in list ---
List:   [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Target: 15
Target: 15
Result: Index -1

--- Target in empty list ---
List:   []
Target: 5
Result: Index -1
PS C:\Users\pende\OneDrive\Desktop\wt2>
```

OBSERVATION:



TASK-3:

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

• Task:

o Use AI to suggest the most efficient search and sort algorithms for this use case.

o Implement the recommended algorithms in Python.

o Justify the choice based on dataset size, update frequency, and performance requirements.

• Expected Output:

o A table mapping operation → recommended algorithm → justification.

o Working Python functions for searching and sorting the inventory.

PROMPT:

Scenario: A retail inventory system manages thousands of products, each with a product ID, name, price, and stock quantity. Staff need to perform two main operations:

1. Quickly search for a product by its unique ID or by its name.
2. Sort the entire product list by price or stock quantity for analysis.

Task:

1. Create a markdown table that recommends the most efficient algorithm for each operation (Search by ID, Search by Name, Sort).
2. For each recommendation, provide a clear justification based on performance characteristics (like time complexity), dataset size, and the specific requirements of the operation.

CODE:

merge_sort.py > merge_sort

```python
def merge_sort(arr: list[dict], key: str) -> list[dict]:

    # Base case: A list with 0 or 1 element is already sorted.
    if len(arr) <= 1:
        return arr

    # 1. Divide: Find the middle of the list and split it into two halves.
    mid = len(arr) // 2
    left_half: list[dict] = arr[:mid]
    right_half: list[dict] = arr[mid:]

    # Recursively sort both halves.
    sorted_left = merge_sort(left_half)
    sorted_right = merge_sort(right_half)

    # 2. Conquer: Merge the sorted halves.
    sorted_arr = []
    i = j = 0  # Pointers for left_half and right_half

    while i < len(sorted_left) and j < len(sorted_right):
        if sorted_left[i][key] < sorted_right[j][key]:
            sorted_arr.append(sorted_left[i])
            i += 1
        else:
            sorted_arr.append(sorted_right[j])
            j += 1
```

```python
    sorted_arr.extend(sorted_left[i:])
    sorted_arr.extend(sorted_right[j:])

    return sorted_arr

# --- Verification with Test Cases ---
if __name__ == '__main__':
    # Sample inventory data (list of dictionaries)
    inventory = [
        {"product_id": 101, "name": "Laptop", "price": 1200, "quantity": 50},
        {"product_id": 102, "name": "Keyboard", "price": 75, "quantity": 200},
        {"product_id": 103, "name": "Mouse", "price": 25, "quantity": 500},
        {"product_id": 104, "name": "Monitor", "price": 300, "quantity": 100},
        {"product_id": 105, "name": "Headphones", "price": 100, "quantity": 150},
    ]

    test_cases = {
        "Sort by Price": ("price"),
        "Sort by Quantity": ("quantity"),
    }

    print("--- Merge Sort Implementation Verification ---")
    for name, key in test_cases.items():
        print(f"\n--- {name} ---")
        print(f"Original: {inventory}")
        sorted_case = merge_sort(inventory,key)
        print(f"Sorted:   {sorted_case}")
```

merge_sort.py > merge_sort

```python
        print(f"Sorted:   {sorted_case}")

def search_by_id(inventory: list[dict], product_id: int) -> dict | None:
    """
    Searches for a product in the inventory by product_id using linear search.
    Assumes the inventory is not sorted by product_id.
    """
    for product in inventory:
        if product["product_id"] == product_id:
            return product  # Return the product if found
    return None  # Return None if not found

def search_by_name(inventory: list[dict], product_name: str) -> dict | None:
    """Searches for a product by name using a dictionary (hashing)."""
    inventory_dict = {product["name"]: product for product in inventory}
    return inventory_dict.get(product_name)

# Example Usage
product = search_by_name(inventory, "Laptop")
if product:
    print(f"\nProduct found: {product}")
else:
    print("\nProduct not found.")
```

OUTPUT:

```
PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/
Desktop/wt2/merge_sort.py
--- Merge Sort Implementation Verification ---

--- Sort by Price ---
Original: [{'product_id': 101, 'name': 'Laptop', 'price': 1200, 'quantity': 50}, {'product_id': 102, '
name': 'Keyboard', 'price': 75, 'quantity': 200}, {'product_id': 103, 'name': 'Mouse', 'price': 25, 'q
uantity': 500}, {'product_id': 104, 'name': 'Monitor', 'price': 300, 'quantity': 100}, {'product_id':
105, 'name': 'Headphones', 'price': 100, 'quantity': 150}]
```

OBSERVATION:

1. **Algorithm Choice**: The selection of algorithms aligns with the requirements. Binary search would be more efficient if the list was sorted by ID, otherwise a linear search is more appropriate. Using hash tables for name-based search provides near-instant lookups. Merge sort provides a stable and efficient sorting mechanism.
2. **Code Quality**: The code is well-documented, with clear explanations of the algorithms and their complexities. The use of type hints enhances readability.
3. **Adaptability**: The `merge_sort` function has been modified to accept a `key` parameter, enabling sorting based on different attributes of the product dictionaries.
4. **Scalability**: The chosen algorithms are suitable for large datasets. Binary search and hashing provide efficient lookups, and merge sort scales well for sorting.
5. **Testability**: The test cases in `merge_sort.py` now verify the sorting by different keys. Example usage of the search functions is also demonstrated.